

ARK-5 Team Task Sheet

Autonomous Navigation -- Python / YAML only

Hardware freeze: ESP32 firmware, wiring, motors, MPU, ultrasonics, ESP32-CAM stay as they are.

1. Direct answers (read this first)

Q: In these tasks, do we only change Python / YAML / laptop side?

YES. Every task in this PDF is laptop-side only: Python files under `human_detector_robot/`, plus optional tweaks in `config/project.yaml`. Do NOT flash ESP32, do NOT edit `.ino` / `config.h`, do NOT add LiDAR, encoders, or extra sensors. Use the data that is already arriving over UDP + camera stream.

Q: Existing hardware only?

YES. Same two robots: ESP32 DevKit (motors + MPU6050 + HC-SR04 ultrasonics) + ESP32-CAM + i5 10th-gen laptop brain. No AMCL / Gmapping / Cartographer -- those need LiDAR. What we WILL add in Python: sonar filtering, better pose, occupancy grid, A* on that grid, corridor centering, unique human pins, swarm wiring fixes.

Q: Sonar kya hai? Ultrasonic sensors ko kaha hai kya?

YES -- in this project, 'sonar' means the ultrasonic distance sensors (HC-SR04). They are the same thing for us.

SONAR originally = SOund Navigation And Ranging (sound ping, listen for echo, $\text{distance} = \text{speed of sound} \times \text{time} / 2$). An ultrasonic sensor does exactly that with a high-frequency beep humans cannot hear. In robotics papers and this codebase, the words 'sonar', 'ultrasonic', and 'HC-SR04' are used interchangeably.

In ARK-5 code you will see names like `dist_front_cm`, `dist_left_cm`, `dist_right_cm`, `wall_near`, and comments saying 'sonar sample'. Those values ARE the ultrasonic readings (centimetres). We do NOT have a LiDAR. When a task says 'filter sonar' it means: clean the ultrasonic numbers in Python before the FSM uses them.

Typical mounts on each robot: Front ultrasonic (stop / creep / scan), Left ultrasonic (junction / corridor), Right ultrasonic (junction / corridor -- already in telemetry; swarm code currently ignores it). Range useful for code: about 15 cm to 300--400 cm. Values ≤ 0 or -1 mean 'no echo / invalid' -- never treat those as a real wall or a real open path.

2. What is already OK (do not debug hardware)

- Ultrasonic (sonar) distances and MPU yaw are transferring over UDP.
- ESP32-CAM stream and YOLO human detection on the laptop work.
- The gap is how Python USES that data for navigation, mapping, and rescue logs.

3. Problems blocking a proper demo

- Pose is fake: map integrates commanded speed x time, not real motion.
- Raw sonar (-1, 2 cm spikes) goes straight into the FSM -- wrong turns / stuck creep.
- No real path planner: wall -> rotate -> pick largest opening. No memory of visited space.
- Cruise does not center in a corridor using left/right ultrasonics.
- Same human marked many times (rescue map looks unprofessional).
- Swarm: `dist_right` forced to -1, WiFi alerts disabled, collision uses drifted (x,y).
- Junction logic: left-always-wins and early-exit before a full scan.

4. Allowed vs forbidden

ALLOWED

- `human_detector_robot/**/*`.py (new `utils/*`.py files are encouraged)
- `human_detector_robot/config/project.yaml` (thresholds, speeds, map size)
- Optional: HUD / ASCII map text output for viva

FORBIDDEN

- esp32/** (.ino, config.h) -- no firmware flash
- wiring changes, extra LiDAR / encoders / Raspberry Pi
- ROS AMCL, Gmapping, Cartographer -- not supportable on this sensor set

5. Five domains

D1 Pose & Sensor Clean -- make ultrasonic + yaw usable

D2 Occupancy Map -- free / wall / unknown grid from sonar rays

D3 A* + Frontiers -- shortest path on that grid (CPU-light on i5)

D4 Local Drive (FSM) -- follow path, junctions, no stuck

D5 Rescue + Swarm -- unique humans, WiFi as hint, two-robot wiring

Work order: D1 -> D2 -> D3 -> D4. D5 can start after D1 in parallel.

6. Tasks (assign one person per task group)

TASK-01 -- Sonar (Ultrasonic) Median Filter

Domain: D1

Problem: Invalid -1 / 2 cm / sudden 400 cm spikes cause false stop or false open-path.

Solution: Keep last 5 samples per sensor; median; ignore ≤ 0 ; reject jumps > 80 cm per sample.

Files: NEW utils/sensor_filter.py; hook in main_controller.py and swarm_controller.py

Depends on: None

Done when: FSM and HUD use filtered cm; raw -1 never treated as a wall or an opening.

TASK-02 -- Yaw Jump Guard

Domain: D1

Problem: Log showed yaw jump 0 deg to -173 deg in one second -- pose and turns explode.

Solution: Unwrap angle; cap change to about ± 90 deg per second; ignore teleport spikes (keep last good yaw). Python only.

Files: utils/sensor_filter.py

Depends on: None (pair with TASK-01)

Done when: Stationary robot: yaw may drift slowly, never teleport.

TASK-03 -- Honest Dead Reckoning

Domain: D1

Problem: Map position moves from commanded v even when wheels are not advancing.

Solution: Update x,y only if failsafe is off, $|v| > 0$, AND front sonar is consistent with moving (distance should shrink when going forward). Freeze pose when $v = 0$.

Files: utils/tentative_map.py (update_pose)

Depends on: TASK-01, TASK-02

Done when: During ALERT_PAUSE the (x,y) pin does not walk across the map.

TASK-04 -- Occupancy Grid from Ultrasonics

Domain: D2

Problem: ASCII dots are not a planner map; wall cells are painted inaccurately.

Solution: 5 cm cells: along each valid sonar ray mark FREE; at the hit mark OCCUPIED (only if 15-300 cm). Front + left + right. Keep UNKNOWN separate.

Files: NEW utils/occupancy_grid.py (or extend tentative_map.py)

Depends on: TASK-01, TASK-03

Done when: Wall glyphs sit at sonar range, not under the robot.

TASK-05 -- Visited Cells + Frontiers

Domain: D2

Problem: Robot can loop the same corridor forever.

Solution: Mark robot cell VISITED. Unknown cells next to free/visited = FRONTIER. Next exploration goal = nearest frontier.

Files: occupancy module from TASK-04

Depends on: TASK-04

Done when: After covering a hallway the next goal is a new opening, not the start.

TASK-06 -- A* Planner

Domain: D3

Problem: No academic path planning -- only rotate-and-guess.

Solution: A* on occupancy grid, 4 or 8 connected, inflate occupied by 1 cell. Output waypoints. If no path, fall back to

current FSM scan. Tiny grid -- fine on i5.

Files: NEW utils/astar.py

Depends on: TASK-04

Done when: At a T-junction A* chooses a corridor on the grid instead of a random spin.

TASK-07 -- FSM: Path Follow + Junctions

Domain: D4

Problem: Left-always-wins, scan early-exit, CREEP stuck (~50 cm front, spinning).

Solution: Cruise follows A* waypoint heading via yaw error -> omega. Corridor centering: (left-right) sonar -> small omega. WIFI_SCAN checks BOTH sides before turning. SCAN_ROTATE completes +/-90 deg (no early-exit). CREEP aborts if 30-40 cm not reached in ~3 s. Align ~4 deg not 8.

Files: utils/autonomous_fsm.py (+ yaml control: keys if needed)

Depends on: TASK-01; TASK-06 if ready, else centering+junction first

Done when: Robot explores without infinite creep; junctions pick the better open side after a full look.

TASK-08 -- Stuck / Oscillation Watchdog

Domain: D4

Problem: Left-right wobble and state ping-pong.

Solution: If pose almost unchanged for ~4 s while a wall is present: force turn opposite to exploration bias. If same FSM state >8 s: reset to CRUISE.

Files: utils/autonomous_fsm.py or NEW utils/watchdog.py

Depends on: TASK-03

Done when: No multi-minute freeze in CREEP_TO_SCAN.

TASK-09 -- Unique Human Tracker

Domain: D5

Problem: One person logged 18 times at nearly the same coordinate.

Solution: New pin only if no existing track within ~80 cm. Priority standing > fallen > wifi. Cooldown is spatial, not only time. Store id, class, conf, x, y, timestamp.

Files: NEW utils/human_tracker.py; change mark_human calls in main_controller.py (and swarm)

Depends on: TASK-03

Done when: One physical person ~ = one map pin.

TASK-10 -- WiFi RSSI as Hint, Not Brake

Domain: D5

Problem: RSSI drop can halt the robot / false-mark humans.

Solution: WiFi only writes '?' on map + log. Stop motors for WiFi only if YOLO also confirms, OR 3 consecutive drops AND a wall in front. Keep empty-room baseline as a flag in yaml / CLI.

Files: utils/wifi_sensing.py, autonomous_fsm.py, main_controller.py, project.yaml

Depends on: TASK-09 recommended

Done when: RSSI flicker does not freeze navigation.

TASK-11 -- Swarm Wiring Fix

Domain: D5

Problem: swarm_controller.py passes dist_right=-1 always and wifi_alert=None -- Robot 2 thinks right is blocked.

Solution: Pass real dist_right from telemetry; enable WiFi like single-robot mode; yield opposite heading for max ~1.5 s then replan; merge both occupancy grids into swarm_combined_map.

Files: swarm_controller.py only

Depends on: TASK-01, TASK-04, TASK-09

Done when: Robot 2 can take a right opening; both maps merge.

TASK-12 -- Demo HUD / Viva Output

Domain: D5

Problem: Logs and ASCII map do not look like a finished FYP.

Solution: HUD: filtered ultrasonics, yaw, FSM state, A* path cells, unique human count, frontier goal. Map legend: wall, path, robot, human, wifi-hint, planned path.

Files: utils/terminal_hud.py and map render

Depends on: TASK-04, TASK-06, TASK-09

Done when: Supervisor can read state + humans + planned path from one screen.

7. Who does what

P1 Filters: TASK-01, TASK-02

P2 Map/pose: TASK-03, TASK-04, TASK-05

P3 Planner: TASK-06

P4 Motion: TASK-07, TASK-08

P5 Rescue: TASK-09, TASK-10

P6 Swarm+demo: TASK-11, TASK-12

If only 2 people: Person A = 01-05, Person B = 07-08, then together 06 and 09-12.

8. How to test without hardware debugging

- Use existing debug/debug_robot_sensors.py only to CONFIRM numbers still arrive.

- After a run, read maps/robot1_movement_log.txt: no yaw teleport, no human spam, no infinite CREEP.
- Do not retune ESP PID or rewire ultrasonics as part of these tasks.

9. One-line summary for the team

Sensing is done. We are not adding SLAM hardware. We will use ultrasonic (sonar) + MPU yaw + YOLO in Python: filter -> occupancy grid -> A frontiers -> better FSM -> unique rescue pins. That is the professional version of THIS robot, not a NUS LiDAR stack.*

File location: project root TEAM_NAVIGATION_TASKS.pdf

Code roots: human_detector_robot/utils/autonomous_fsm.py (navigation FSM)

human_detector_robot/utils/tentative_map.py (pose + ASCII map)

human_detector_robot/main_controller.py (single robot loop)

human_detector_robot/swarm_controller.py (two robots)

human_detector_robot/config/project.yaml (laptop config only)